

LC 1106: Parsing A Boolean Expression

Authored by: Arpit Raj, LNMIIT Jaipur | Date: August 31, 2026

Amazon

Hilabs

1. Examples & Dry Runs

EXAMPLE 1

Expression: `&( |(f) )`

- Step 1: `| (f) → f`
- Step 2: `&(f) → f`

Output = false

EXAMPLE 2

Expression: `| (f, f, f, t)`

Process: `f + f + f + t = t`

Output = true

EXAMPLE 3

Expression: `!( &(f, t) )`

Inner Calculation: `(f and t) = (0 and 1) = 0 = false`

But there's a ! outside, which makes it **true**.

KEY OBSERVATIONS

- `&(x, y, z)` → it's **false** if *any false*.
- `|(x, y, z)` → it's **true** if *any true*.
- `!(x)` → **inverse**.

2. Approach & Stack Progression

Let's consider the expression `&( t, |(f, t), !(f) )`.

The outer `&` can't be evaluated until `|(f, t)` and `!(f)` are completely evaluated.

So, we evaluate the inner expression → calculate → give result to the outer expression. **Hence, we use a Stack.**

STACK PROGRESSION DRY RUN

Let the expression be: `&( t, |(f, t) )`

```
[ ] → [&] → [&, (] → [&, (, t] → [&, (, t, |]  
Now encounter '(', then push inner elements:  
→ [&, (, t, |, (, f, t]
```

Now we encounter `)`, so we pop until `(`.

Pop `f` and `t`. For the `|` operator → `f + t → t`

Basically `|(f, t) → t`

So the Stack becomes: `[&, (, t, t]`

3. Algorithmically

Let's evaluate the state `[&, (, t, f, t]`. Now we see a `)`.

- `t` → `hasTrue = true`
- `f` → `hasFalse = true`

- `t` → `hasTrue = true`

Now we reach `(` and stop. We pop `(`.

Stack is now `[&]`. We pop it → Operator is `&`.

Since the operator is `&` and `hasFalse = true`, the result = **false**.

So we push the result to the Stack → Stack = `[f]`.

4. C++ Implementation

```

class Solution {
public:
    bool parseBoolExpr(string expression) {

        stack<char> st;
        int n = expression.size();

        for (int i = 0; i < n; i++) {
            char ch = expression[i];
            if (ch == ',')
                continue;

            else if (ch == '(' || ch == '!' || ch == 't' || ch == 'f' || ch ==
'|' ||

                    ch == '&') {
                st.push(ch);
            }

            else { // close bracket

                bool hastrue = false;
                bool hasfalse = false;

                while (st.top() != '(') {
                    char c = st.top();
                    st.pop();

                    if (c == 't')
                        hastrue = true;
                    else if (c == 'f')
                        hasfalse = true;
                }

                st.pop(); //( isko hatane ke liye

                char op = st.top();
                st.pop();
                bool result;

                if (op == '&') {
                    if (hasfalse == true)
                        result = false;
                    else
                        result = true;
                }
            }
        }
    }
};

```

```
        else if (op == '|') {
            if (hastrue == true)
                result = true;
            else
                result = false;
        }

        else { //not operator
            if (hastrue == true)
                result = false;
            else if (hasfalse == true)
                result = true;
        }

        if(result) st.push('t');
        else st.push('f');
    }
}

return st.top() == 't';
}
};
```